PROJECT DOCUMENTATION:

LIBRARY MANAGEMENT APPLICATION

Name: Barboi Ionela Madalina

Project Name: Library App

Date: January 7, 2026

1.Project abstract

The Library App is a desktop application designed to help users keep track of their personal reading habits and stay connected with friends. It allows readers to monitor their progress page by page, follow other users, and take part in reading challenges. The app focuses on a simple and friendly user experience. It is developed using Java (JavaFX) for the user interface and PostgreSQL for database management, and it follows Object-Oriented Programming principles

How to run the application:

Before running the project make sure you have the following installed:

- Java Development Kit JDK version 21 or higher
- PostgreSQL database version 14 or higher
- PgAdmin 4 or any SQL client
- IntelliJ IDEA Community or Ultimate
- Scene Builder optional but recommended for UI editing

Database Setup

The app will not start without the database.

- Open PgAdmin 4
- Create a new database called library_app
- Open the Query Tool for this database
- Run the SQL scripts in this order.
- First run schema.sql this creates tables relationships and triggers
- Second run sample_data.sql this adds users books and reviews so the app is not empty
- Update database credentials.
- Go to src main java org andrei sample_project connection ConnectionFactory.java
  Change the USER and PASSWORD values to your local PostgreSQL credentials

External Libraries
The project uses these dependencies.

- JavaFX javafx controls and javafx fxml for the interface
- PostgreSQL JDBC driver org postgresql postgresql
- JBcrypt org mindrot jbcrypt for password hashing

These are handled with Maven .

Modifying the Interface

- The UI is built with FXML files located in src main resources. You do not need to write XML by hand.
- Open any fxml file with Scene Builder.
- You can drag and drop elements move buttons resize tables and change colors visually. After saving make sure fx id values and onAction methods match the controller classes.

Running the Application

- Go to src main java org andrei sample_project LibraryApplication.java
- Run the main method.
- Test login accounts.
  Admin username admin password password123
  User username sarah_books password password12

2.Implemented features

The application is built around two main types of users: Readers (Users) and Administrators. Each role has a different experience inside the app, and the navigation changes automatically based on the user's role and privacy settings.

Password Hashing: Passwords are encrypted using BCrypt inside the UserRepository, so they are never stored as plain text.

Role-Based Access Control : After login, the system identifies whether the user is an ADMIN or a USER and redirects them to the appropriate dashboard (MainController for users, AdminController for admins).

Database Connectivity: The app uses PostgreSQL for data storage, supporting complex queries, joins, and transaction management.

The User Experience (Reader Flow)

The User dashboard is the heart of the application and is designed to be interactive and engaging.

Users manage their reading through three automatically updated lists:

To Read: Books the user plans to read in the future.

Currently Reading: Books that are in progress.

Users can update the current page, and the system automatically calculates the reading percentage (e.g., *Page 150/300 – 50%*).

When the final page is entered, the book is automatically moved to Completed, and a popup asks the user to leave a review.

Completed: A history of books the user has finished.

Social Features

The app also includes a social component that allows users to connect with others.

User Search: Users can search for others by username or full name. You can also search for books or authors, or all using the search filter.

Privacy Logic:

Following a public account happens instantly.

Following a private account sends a follow request with a *PENDING* status, which must be accepted before access is granted.

Profile Viewing:

Users can see other profiles with statistics such as books read, pages read, recent reviews, favorite books, who the other person is following and who the other person is followed by.

If a profile is private and not followed, sensitive reading data remains hidden.

Author's pages can be accesed too (they include information about the writer and a

table with author's books.

Users can see their own profile, edit their name and bio.(future implementations:

Toggle privacy(can change acount privacy after creating the account), Add profile picture)

To motivate users, the app includes gamification features.

Community Challenges: Users can join global challenges created by admins (e.g., *"Summer Sci-Fi Reading"*).

Automatic Progress Tracking: When a book is completed, the system checks if it fits the challenge rules (genre, date range, etc.) and updates progress automatically.

Badges: Completing a challenge rewards the user with a digital badge displayed on their profile.

Personal Goals: Users can set personal reading goals (like *"Read 20 books this year"*) and track progress with a visual progress bar.

Reviews: Users can rate books from 1 to 5 stars and write reviews, which contribute to the book's overall rating.

Favorites: Users can mark books as favorites for quick access later.

The Admin Experience (Administrator Flow)

The Admin Panel is focused on managing content and maintaining the system.

Content Management (CRUD Operations)

Books: Admins can add, edit, or delete books.

Images: Book cover images can be uploaded and stored in a local resource folder.

Authors: Admins manage author information such as biography, dates, and nationality. Deleting an author checks for linked books to avoid database issues.

Reviews: Inappropriate reviews can be moderated or removed.

Challenge Creation

Admins define reading challenges by setting: the number of books required, allowed genres (specific or any), start and end dates ,badge name and icon.

Analytics Dashboard

The admin dashboard displays overall system statistics, such as:

- Total registered users
- Total books in the library
- Active challenges
- General activity data (e.g., number of reviews)

3.Future implementations:

One of the main improvements would be a full image and visual management system. While I attempted to implement image uploads from a local computer, this feature was not fully functional, so currently the application only displays a default book cover.

In a future version, I would like to implement**:**

- Cloud or BLOB Storage: Instead of using local file paths, images would be stored either directly in the PostgreSQL database as BLOBs (Binary Large Objects) or using a cloud

storage service such as AWS S3 or Cloudinary. This would solve persistence and portability issues.

- User Profile Pictures: At the moment, user profiles display a text-based avatar generated from initials (for example, *"JD"* for John Doe). Allowing users to upload custom profile pictures would make profiles more personal.
- Author Portraits: The Author entity currently supports biographies and dates only. Adding author photos would improve the overall presentation and make the author details page more engaging.

2. Social Feed & Activity Log

Another planned improvement is transforming the current static dashboard into a dynamic social feed.

- When a followed user finishes a book or writes a review, this activity would appear in the feed (e.g., *"Sarah just rated 'Dune' 5 stars"*).
- This would require introducing a new ActivityLog table in the database to store user actions along with timestamps.
- The feed would show recent activity from followed users, making the app feel more social and interactive.

3. Book Recommendation System

Using existing data from tables such as User_Books and Reviews, a basic recommendation system could be implemented.

- The system would suggest books based on genres, ratings, and reading history.
- Example logic: *"Because you read 'Harry Potter' (Fantasy) and rated it 5 stars, you might enjoy 'Percy Jackson'."*

4.Data Base Schema:

Database schema can be found in sample_data.sql. Sample data for this schema can be found in sample_data.sql.

```
users ──< user_books >── books ──< reviews >── users
│ │
│ └──< coming_soon_books
│
├──< followers (self-reference)
├──< personal_challenges ──< personal_challenge_books >── books
└──< user_app_challenges >── app_challenges

authors ──< books
```

Entity Relationship Summary

Table	Primary Key	Foreign Keys	Relationships
users	user_id	-	1→M user_books, 1→M reviews, 1→M personal_challenges, 1→M user_app_challenges, self M↔M followers
authors	author_id	-	1→M books
books	book_id	author_id	M→1 authors, 1→M reviews, M↔M users (via user_books), M↔M personal_challenges (via personal_challenge_books)
user_books	user_book_id	user_id, book_id	M→1 users, M→1 books
reviews	review_id	user_id, book_id	M→1 users, M→1 books
followers	(follower_id, following_id)	follower_id, following_id	Self M↔M users
coming_soon_books	coming_soon_id	book_id	M→1 books
app_challenges	app_challenge_id	-	1→M user_app_challenges
user_app_challenges	user_app_challenge_id	user_id, app_challenge_id	M→1 users, M→1 app_challenges
personal_challenges	personal_challenge_id	user_id	M→1 users, 1→M personal_challenge_books
personal_challenge_books	personal_challenge_book_id	personal_challenge_id, book_id	M→1 personal_challenges, M→1 books

5.Java files:

1. ConnectionFactory.java

This class manages the connection between the Java application and the PostgreSQL database. I used the Singleton pattern concept here to ensure we don't open too many

connections at once. It loads the database driver and handles any errors if the database is offline or the credentials are wrong.

2. LibraryApplication.java

This is the main entry point of the program. It launches the JavaFX application and loads the initial login-view.fxml screen. It sets the primary stage (window) dimensions and title.

3. LoginController.java

This controller handles the logic for the login and registration screens. It checks if the user is an Admin or a regular User and redirects them to the correct dashboard. I also implemented the registration logic here, including the toggle for "Public" vs. "Private" accounts.

4. MainController.java

This is the core controller for the User Dashboard. It manages the three main tabs (Reading, To Read, Completed) and handles the search functionality. It includes the logic for updating reading progress when a user finishes a book, this controller automatically moves it to the "Completed" list.

5. AdminController.java

This controller powers the Admin Dashboard. It allows administrators to perform CRUD operations (Create, Read, Update, Delete) on books, authors, and reviews. It also includes the logic for creating global challenges and viewing system-wide statistics.

6. UserProfileController.java

This class manages the view when you look at *another* user's profile. It contains the logic to check privacy settings: if a user is private and I don't follow them, it hides their stats. It also handles the "Follow" button, changing it to "Requested" or "Following" based on the database status.

7. ProfileController.java

This controls the *current* user's personal profile page. It calculates and displays their reading stats (books read, badges earned) and allows them to edit their bio. It connects to the FavoritesDialog to show their top books**.**

8. ChallengeController.java

This controller handles the gamification side of the app. It displays the "Community Challenges" created by admins and allows users to create their own "Personal Challenges." It dynamically updates progress bars based on how many books the user has read.

9. AuthorController.java

This class controls the view that displays an Author's details. It shows their biography, a calculated "lifespan" (Birth - Death), and a list of all books written by them. I added a fallback here so if the author has no photo, it shows their initials.

10. AuthorEditController.java

This is used by Admins to update author information. It pre-fills the form with existing data (like name and birth date) and allows the admin to save changes back to the database.

11. User.java (Model)

This model represents a user account. It stores critical information like the isPrivate flag (for privacy settings) and the role (Admin vs. User). I added a helper method getInitials() to generate an avatar if they don't have a profile picture.

12. Book.java (Model)

This class represents a book. It stores metadata like title, genre, and rating. It includes a getDefaultCoverImage() method that acts as a safety net if a cover image fails to load, it provides a default one so the app doesn't crash.

13. Author.java (Model)

This model holds author data. I added a helper method called getLifeSpan() that formats the birth and death dates into a readable string (e.g., "1920 - 1992") for the UI.

14. Review.java (Model)

This represents a review written by a user. It links a User ID and a Book ID with a rating (1-5) and text. It includes a method to convert the integer rating into a string of stars.

15. AppChallenge.java (Model)

This represents the global challenges created by admins. It contains business logic to calculate the percentage of completion based on the user's progress versus the target.

16. PersonalChallenge.java (Model)

Similar to AppChallenge, but for user-created goals. It includes dates (start/end) and tracks how many books the user has read towards their personal goal.

17. UserAppChallenge.java (Model)

This is a "junction" model that tracks a specific user's progress on a specific global challenge. It links the User, the Challenge, and the Badge they earn upon completion.

18. FollowRequests.java (Model)

A simple model class to manage the state of social connections. It tracks the requester, the target user, and the status (PENDING, ACCEPTED, DECLINED).

19. ComingSoonBook.java (Model)

This represents unreleased books shown on the dashboard. It has a method getStatusDisplay() that converts database codes (like "UPCOMING") into user-friendly text with emojis.

20. BookDialog.java (Helper)

This is a custom dialog class I built to reduce code repetition. It handles multiple actions for a book: viewing details, rating, reviewing, and moving it between lists. It uses a callback system so the main window refreshes automatically when changes are made.

21. MenuHelper.java (Helper)

This class creates the navigation menu popup. Since the menu is needed on almost every screen, putting it here kept my controllers clean and ensured the navigation looked consistent everywhere.

22. FavoritesDialog.java (Helper)

A specialized dialog that displays a table of the user's favorite books. It allows users to double-click a book to jump straight to its details.

23. FollowRequestsDialog.java (Helper)

This manages the social notifications. For private users, it shows "Accept/Decline" buttons for new followers. For public users, it simply shows a list of who has recently followed them.

24. UserRepository.java

Handles all user database operations. It includes the login logic, register logic (checking for duplicate usernames), and updates privacy settings**.**

25. BookRepository.java

Manages book data. It contains complex SQL queries to join the books, user_books, and authors tables so we can see which books a user is reading and what page they are on.

26. AuthorRepository.java

Handles database operations for authors, including searching for authors by name and retrieving the list of books associated with them.

27. ReviewRepository.java

Manages reviews. It retrieves reviews for specific books and calculates the new average rating for a book whenever a review is added or deleted.

28. ChallengeRepository.java

This class handles the gamification logic. It checks if a user has met the criteria for a challenge (e.g., "Read 5 Fantasy books") and updates their progress in the database.

29. FollowRepository.java

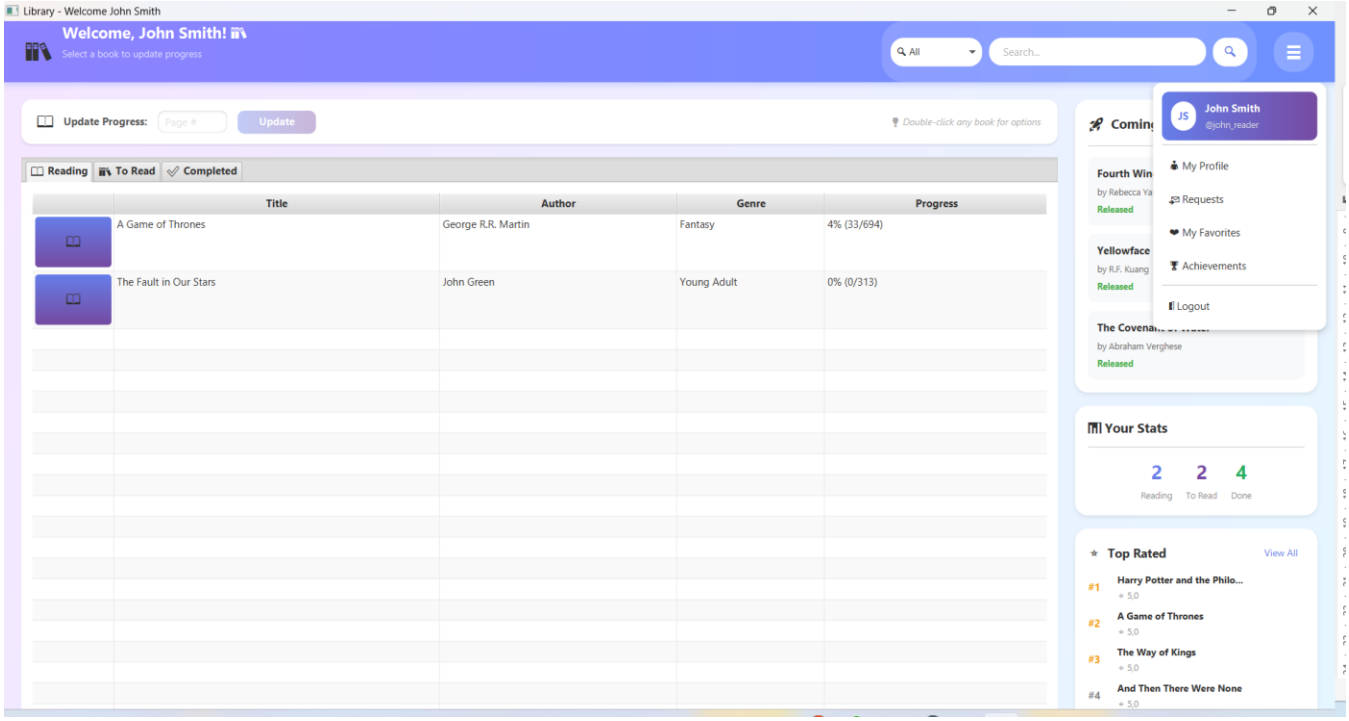
Manages the social graph. It implements the logic for following users: if the target is private, it sets the status to PENDING; otherwise, it sets it to ACCEPTED.

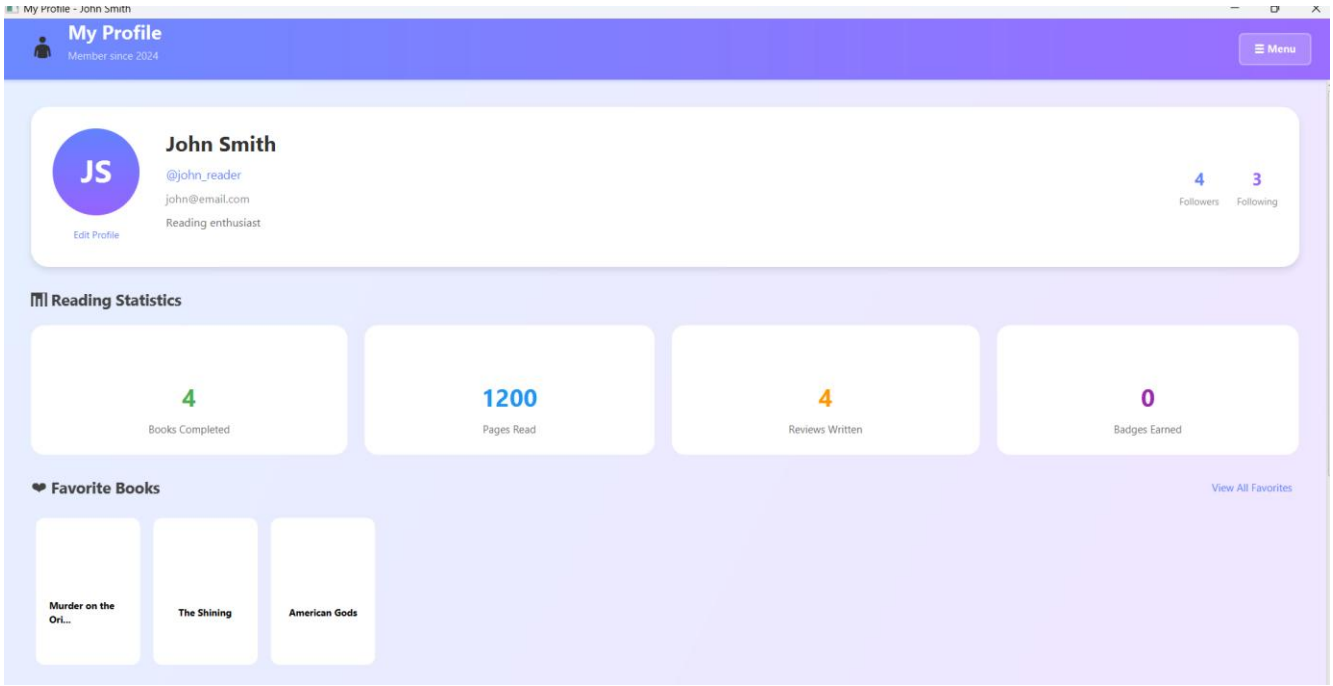
30. FavoriteRepository.java

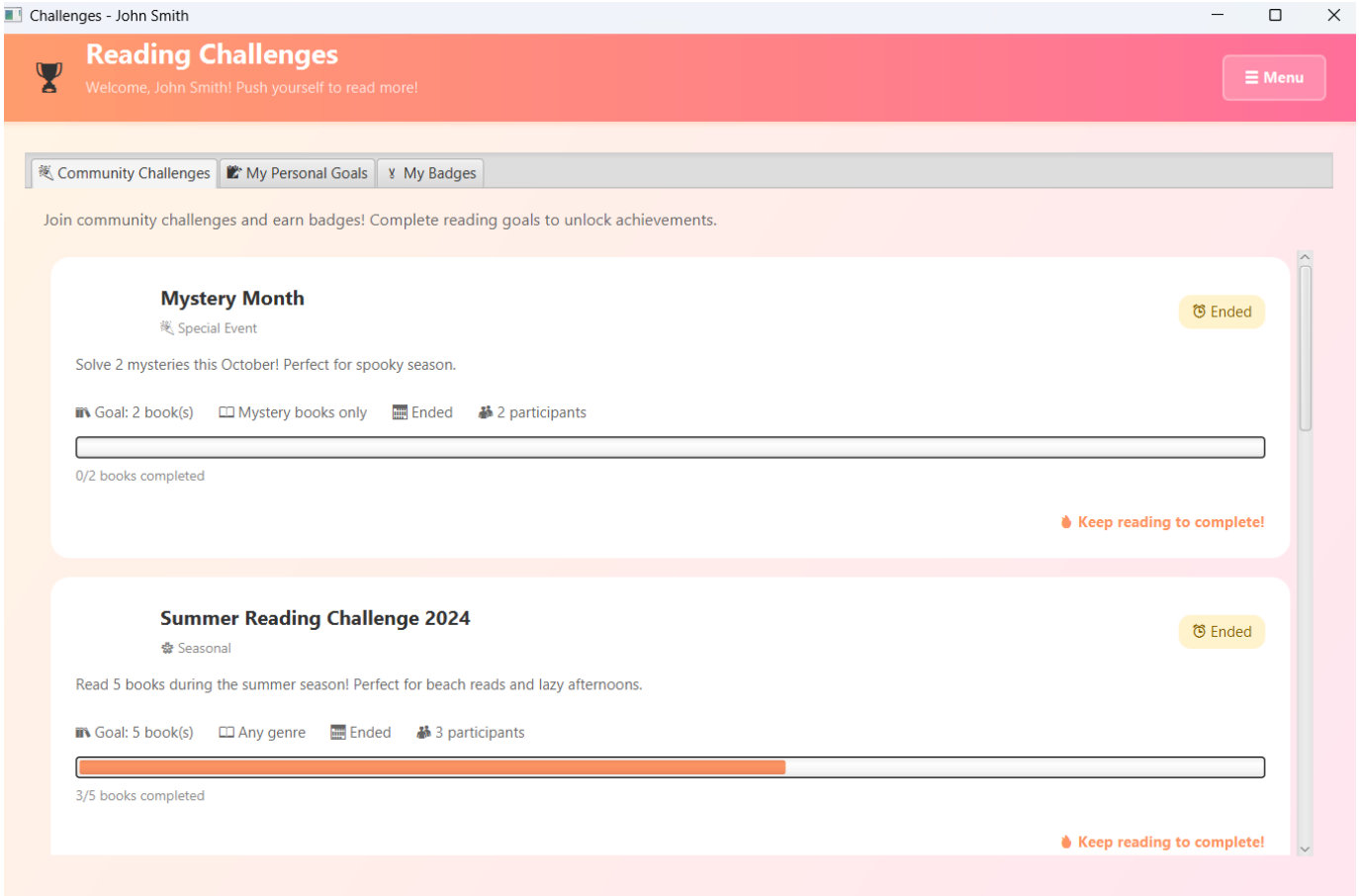
Simple repository to manage the "Favorites" list, allowing users to toggle the heart icon on books to add or remove them from their favorites list.

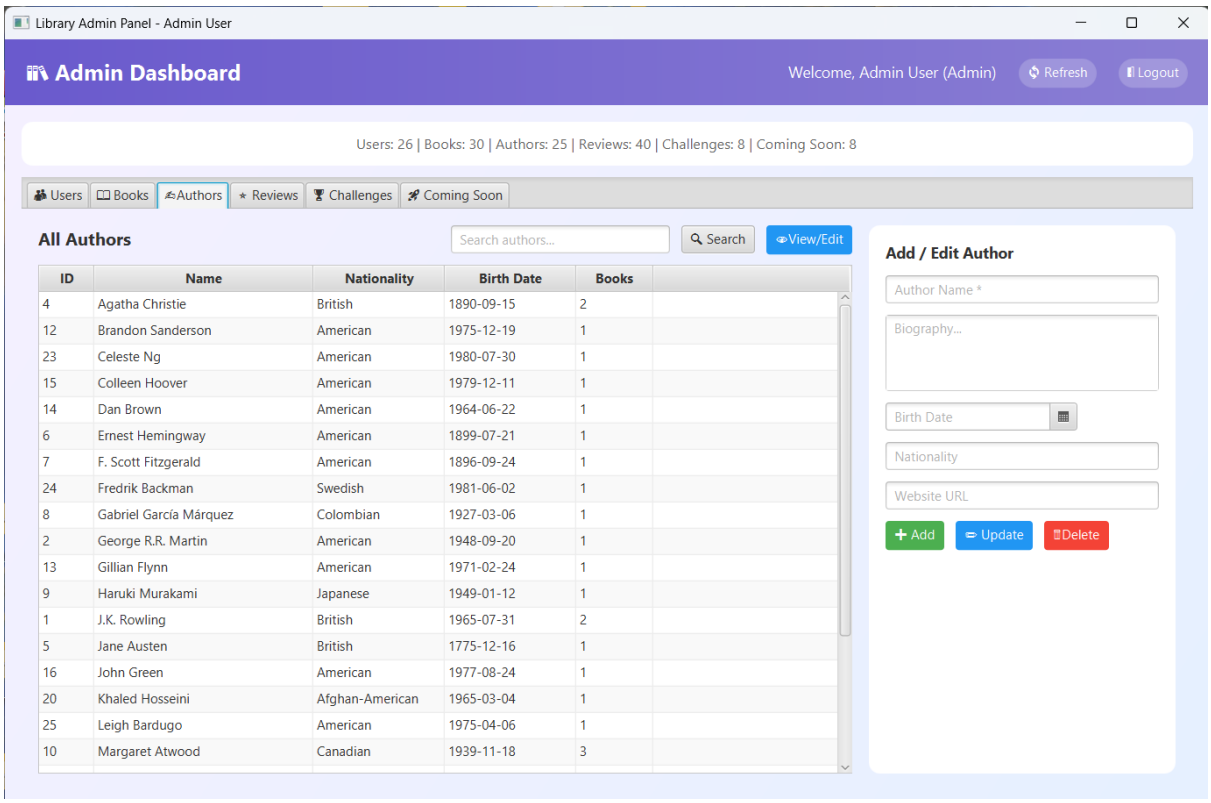
31. ComingSoonRepository.java Fetches the list of upcoming books for the dashboard. It orders them by release date so users see the nearest releases first.

6.UI

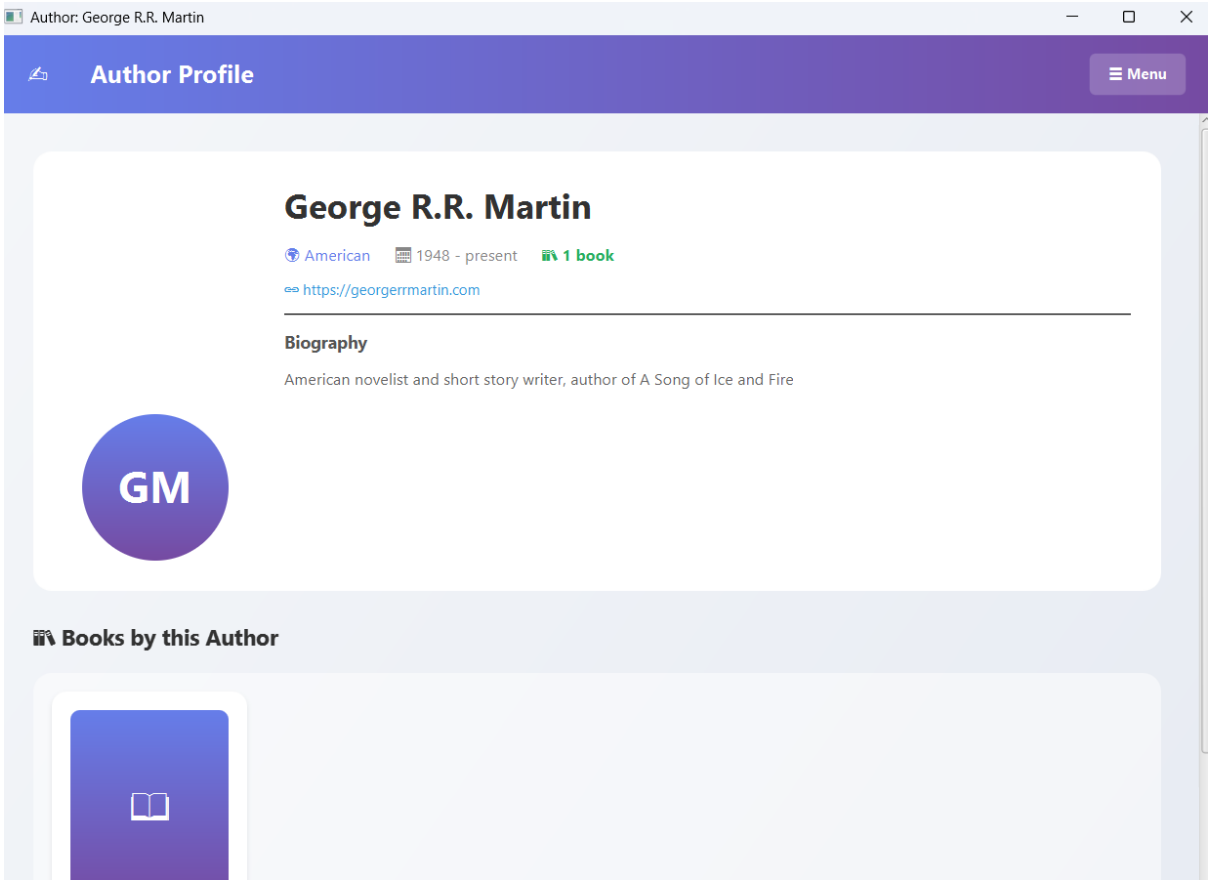

Library - Welcome John Smith
Welcome, John Smith!
Select a book to update progress
All
Search...
Update Progress:
Page #
Update
Double-click any book for options
Reading
To Read
Completed
Title
Author
Genre
Progress
A Game of Thrones
George R.R. Martin
Fantasy
4% (33/694)
The Fault in Our Stars
John Green
Young Adult
0% (0/313)
John Smith
@john_reader
My Profile
Requests
My Favorites
Achievements
Logout
Fourth Win
by Rebecca Ya
Released
Yellowface
by R.F. Kuang
Released
by Abraham Verghese
Released
Your Stats
2
Reading
2
To Read
4
Done
Top Rated
View All
#1
Harry Potter and the Philo...
#2
A Game of Thrones
#3
The Way of Kings
#4
And Then There Were None


My Profile - John Smith
My Profile
Member since 2024
Menu
JS
John Smith
@john_reader
john@email.com
Reading enthusiast
Edit Profile
4
Followers
3
Following
Reading Statistics
4
Books Completed
1200
Pages Read
4
Reviews Written
0
Badges Earned
Favorite Books
View All Favorites
Murder on the Ori...
The Shining
American Gods





ID	Name	Nationality	Birth Date	Books
4	Agatha Christie	British	1890-09-15	2
12	Brandon Sanderson	American	1975-12-19	1
23	Celeste Ng	American	1980-07-30	1
15	Colleen Hoover	American	1979-12-11	1
14	Dan Brown	American	1964-06-22	1
6	Ernest Hemingway	American	1899-07-21	1
7	F. Scott Fitzgerald	American	1896-09-24	1
24	Fredrik Backman	Swedish	1981-06-02	1
8	Gabriel García Márquez	Colombian	1927-03-06	1
2	George R.R. Martin	American	1948-09-20	1
13	Gillian Flynn	American	1971-02-24	1
9	Haruki Murakami	Japanese	1949-01-12	1
1	J.K. Rowling	British	1965-07-31	2
5	Jane Austen	British	1775-12-16	1
16	John Green	American	1977-08-24	1
20	Khaled Hosseini	Afghan-American	1965-03-04	1
25	Leigh Bardugo	American	1975-04-06	1
10	Margaret Atwood	Canadian	1939-11-18	3


Author: George R.R. Martin
Author Profile
Menu
George R.R. Martin
American
1948 - present
1 book
https://georgerrmartin.com
Biography
American novelist and short story writer, author of A Song of Ice and Fire
GM
Books by this Author